

Multi-Variate Temporal GAN for Large Scale Video Generation

Andres Munoz*, Mohammadreza Zolfaghari*, Max Argus, Thomas Brox

University of Freiburg

{amunoz, zolfagha, argusm, brox}@informatik.uni-freiburg.de

Abstract

In this paper, we present a network architecture for video generation that models spatio-temporal consistency without resorting to costly 3D architectures. In particular, we elaborate on the components of noise generation, sequence generation, and frame generation. The architecture facilitates the information exchange between neighboring time points, which improves the temporal consistency of the generated frames both at the structural level and the detailed level. The approach achieves state-of-the-art quantitative performance, as measured by the inception score, on the UCF-101 dataset, which is in line with a qualitative inspection of the generated videos. We also introduce a new quantitative measure that uses downstream tasks for evaluation.

1. Introduction

Generative Adversarial Networks (GANs) [21] have enabled powerful ways to generate high-resolution images [48, 3]. Video generation adds further complexity, as the resulting content should not only make sense spatially but should also be temporally coherent. This is particularly true for the aspect of motion, which does not exist in still images.

3D Convolutional Neural Network (CNN) architectures appear well-suited to trivially lift the progress made in single images to videos [13, 7, 40, 18], yet their usefulness for video generation is still being debated [44, 37]. One argument against 3D CNNs is that the temporal dimension behaves differently from the spatial dimensions. The authors of MoCoGAN [44] showed that equal treatment of space and time results in fixed-length videos, whereas the length of real-world videos varies. Moreover, according to studies in literature [23, 15] 3D CNNs have more parameters, which makes them more prone to overfitting [19].

We share the view of TGAN [37] and MoCoGAN [44], where instead of mapping a single point in the latent space



Figure 1: Our proposed MVT-TSA method produces both high quality video frames and coherent motion. Frame quality is shown by results for UCF-101 comparing (a) MDP [52], (b) MoCoGAN [44] and (c) TGANv2 [42] to our method (d). Motion quality is shown by a sample of frames from MVT-TSA trained on the Jester [24] dataset (e).

to a video, a video is assumed to be a smooth sequence of points in a latent space where each point corresponds to a single frame in the video. As a result, our video generator consists of two submodules: a *sequence generator* that generates a sequence of points in the latent space, and an *image generator* that maps these points into image space.

For the image generator, we propose a Temporal Self-Attention Generator, which introduces a temporal shifting mechanism into residual blocks of the generator. We repurpose the Temporal Shift Module (TSM) proposed by Lin *et*

*Equal Contribution

al. [23] for video understanding, to achieve similar spatio-temporal modeling ability as 3D CNNs. The temporal shifting module is complementary to 2D convolutions in the image generator and allows us to efficiently model the temporal dynamics of a video by facilitating the information exchange between neighbor frames.

In addition to the Temporal Self-Attention Generator, we also introduce a new latent code sampling for the sequence generator. Motivated by MoCoGAN, we re-design the sequence code generator in a way that the latent code is sampled from a multivariate Gaussian distribution. Our sequence generator embeds this latent code using a recurrent network, this allows us to efficiently model the video dynamics and static semantics. We found that this design has a profound effect on the quality and time-consistency of the generated samples.

The growing interest in video generation methods gives rise to challenges in comparing the quality of generated samples. There are two types of evaluation measures: qualitative and quantitative measures. On one side, qualitative measures (e.g. human rating) cannot detect memorization or low diversity. On the other side, quantitative measures are not robust and consistent [4, 5, 34]. Although IS has gained popularity in evaluating the quality of generated images, it has several drawbacks particularly failing to detect mode collapse and memorization. FID assumes that features are from Gaussian distribution, which is not always a valid assumption. Both IS and FID measures are sensitive to pre-processing.

Therefore, we introduce a new evaluation measure named Symmetric-Similarity-Score (S3) to measure the quality of generated videos. S3 measures the domain gap of an action classifier when trained on synthesized videos and tested on real ones, and vice-versa. Consequently, it penalizes missing intra-class diversity, and also is sensitive to both structural deficits and low-level artifacts in the generated data. Hence, it is more robust to over-confident predictions, and it is less sensitive to model parameters or pre-processing. Experiments on the UCF101, Jester, and Weizmann datasets show substantial improvements in the quality of the generated videos for the proposed design compared to previous work.

The paper makes two contributions: (1) it introduces a new 2D video generator design with a strong ability to model spatio-temporal content by facilitating the information exchange between neighboring frames. (2) It introduces a new evaluation metric based on the domain gap between synthesized and real videos in terms of video classification performance.

2. Related Work

Image generation has recently seen leaps in performance, thanks to recently introduced frameworks such as InfoGAN

[55], which learned a more interpretable latent code. SN-GAN (Miayato *et al.* [47]), introduced the concept of spectral normalization of the discriminator’s weights. StyleGAN (Karras *et al.* [48]), leveraged the coarse-to-fine approach introduced in ProGAN [49] to learn to control certain aspects of the image at each generation level in a completely unsupervised manner. Zhang *et al.* [17], designed a self-attention module that allowed the network to create non-local spatial relationships. Then, BigGAN [3] build upon this work by establishing some architectural and training guidelines by which GANs can be stable, converge faster and produce better quality samples. Coupled with the already mentioned benefits, it was shown that by truncating the latent code distribution a trade-off between diversity and quality could be achieved. In addition, DeLiGAN [45], proposed increasing the modelling power of the distribution by representing the latent space with a Mixture of Gaussians model. In this study we propose MVT-GAN, which builds upon BigGAN and extends it to video. MVT-GAN generates videos in a per-frame basis, it can thus exploit further developments on image generation.

Video generation is a highly challenging task as a result of needing to ensure a smooth transition across video frames. Most works in video generation have been on the closely related frame prediction task [12, 54, 57, 25, 50, 8, 38, 41]. The main difference between video generation and frame prediction, is that in frame prediction the network is trying to generate a set of T frames given a set of N previously seen frames. Conversely, video generation only uses the latent code, and in some occasions a label, to generate a set of frames. Frame prediction is an easier problem than video generation due to the strong prior the set of previously seen frames impose on the appearance and motion of the video. Besides frame prediction, there are other closely related tasks to video generation like text-to-video [56, 22] and image-to-video [16, 44]. In this work we focus only on video generation conditioned solely on an class label.

Several frameworks for video generation using GANs have been proposed in the past. Vondrick *et al.* [6] proposed a two-stream network which explicitly separated the generation of the foreground and the background. They assumed that background in the entire video is static, which is not true in real-world video datasets. Saito *et al.* [37] introduced a temporal generator to transform a single latent variable into a sequence of latent variables, to be able to just utilize a 2D network as a generator. As a matter of fact, they showed that a 2D generator can outperform a 3D one. MoCoGAN [44] separated motion and appearance features by dividing the latent code in two smaller sub-codes, one per each set of features. In MoCoGAN, a single content code is sampled from a Gaussian distribution for the entire video which cannot be representative for the whole visual content of the

video. Acharya *et al.* [11] used a coarse to fine approach to improve quality and convergence speed. TGANv2 [42], efficiently trains models that generate high dimensional samples by subsampling features and videos from the batch. DVD-GAN [2] -a concurrent work-, leveraged a high capacity model to synthesize high quality samples from complex datasets. These models showed that GANs can be effective at generating videos. Nevertheless, the previously proposed models either suffer from lack of quality, mode collapse, memorization or require an excessive amount of computational power and data to train properly. Our framework outperforms the state-of-the-art on UCF-101, based on IS measure, while keeping memorization to a minimum. Moreover, previous methods lack a complete quantitative assessment of the performance of the respective methods. They rely on metrics such as IS [43] and FID [20] which don't tell the full story about sample quality. These metrics are dependent on availability of models and are also sensitive to changes in the pipeline. Here we introduce a metric, called Symmetric Similarity (S3), which aims to represent both quality and diversity in a single scalar value. In addition, S3 is robust to changes in pre-processing and model parameters.

3. Generative Adversarial Networks

Before describing our architecture we briefly review GANs. GANs [21] are a class of generative models consisting of a generator and a discriminator networks. The discriminator is a binary classifier that outputs the probability a sample is either real or synthesized. The generator is a function that generates synthetic samples x that look similar to real samples.

GAN training is a minimax game, in which the discriminator D tries to minimize the probability of making a mistake, while the generator G seeks to maximize this probability:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (1)$$

where p_{data} is the distribution of the empirical data and p_z represents the chosen prior distribution of the latent codes z .

The Jensen-Shannon Divergence (JSD) minimization, which is performed in GANs, has been proven empirically to behave more like reverse KL divergence rather than the KL divergence [35]. The reverse KL divergence measures the dissimilarity between two distributions for the fake samples, and there is no penalty on the fraction of the data distribution covered by the generated distribution. This leads to mode collapse in GANs. Different strategies have been proposed to overcome this issue. WGAN [35] pioneered to use Wasserstein distance to avoid this issue. The recent BigGAN method [3] demonstrated state-of-the-art performance

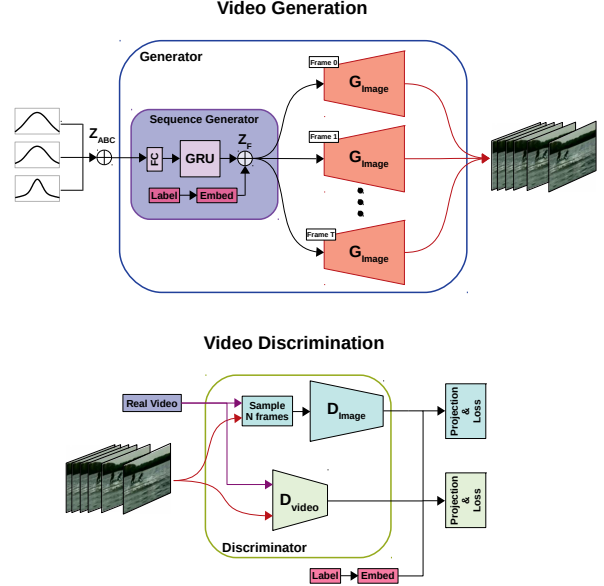


Figure 2: MVT-GAN framework. Sampled Gaussian noise is with several different variances is concatenated ($\oplus$) into Z_{ABC} . A sequence generator using gated recurrent units is then used to generate a vector Z_F . The image generator then transforms this into video frames. Video discrimination is done by a 2D discriminator (D_{Image}) judging a subset of the video frames and a 3D discriminator (D_{Video}) judging all frames and assessing the motion consistency of the video.

in image synthesis by leveraging the best practice of previous methods, such as spectral normalization and projection.

4. Multi-Variate Temporal GAN

The proposed video generation architecture MVT-GAN consists of a sequence generator, an image generator and a video discriminator; an overview of which is shown in Figure 2. It is a projection based conditional GAN approach as proposed by Miyato & Koyama [39] using the hinge formulation of the GAN objective (Lim & Ye [32]; Tran et al. [14]):

$$L_D = \mathbb{E}_{(x,y) \sim p_{data}} [\min(0, -1 + D(x, y))] - \mathbb{E}_{z \sim p_z, y \sim p_{data}} [\min(0, -1 - D(G(z), y))] \quad (2)$$

$$L_G = -\mathbb{E}_{z \sim p_z, y \sim p_{data}} [D(G(z), y)] \quad (3)$$

We introduce several improvements to different aspects of the video generating framework including the noise sampling, sequence generator and the image generator.

4.1. Generator

The generator is divided into two parts. First we generate a sequence of latent codes, then in the second step the image generator maps these latent codes to a sequence of T frames.

4.1.1 Noise Sampling

We construct the latent space $Z_{ABC} \in \mathbb{R}^d$ as 3 independent multi-variate Gaussian distributions $Z_A \in \mathbb{R}^{d_A}$, $Z_B \in \mathbb{R}^{d_B}$ and $Z_C \in \mathbb{R}^{d_C}$ with their diagonal covariance matrices Σ_A , Σ_B and Σ_C respectively.

This allows for the construction of our latent codes, Z_{ABC} ¹, by concatenation of Z_A, Z_B, Z_C as $Z_{ABC} = [Z_A, Z_B, Z_C]^T$. The final distribution Z_{ABC} is a multi-variate Gaussian distribution with diagonal covariance matrix. By adopting an independent parametrization of the subspaces, the network is able to learn more nuanced distributions, thus a better modelling of the features. Subspaces have no prior meaning - the network learns to interpret each part of the code as it sees fit.

4.1.2 Sequence Generator

The latent code Z_{ABC} does not have a temporal dimension. Since our generator is image based, we first have to create a progression of correlated latent codes that extends through the intended duration of the video.

This is done by the sequence generator. It first applies a learnable linear transformation to the latent codes:

$$Z_{fc} = FC(Z_{ABC}), \quad (4)$$

where FC is a fully-connected layer. Then we feed Z_{fc} into a Gated Recurrent Unit (GRU) to generate a sequence of T correlated codes as $Z_{gru} = [z_{gru}^1, \dots, z_{gru}^T]^T$, where each z_{gru}^i corresponds to the i -th frame in the video sequence. In total this results in an input of size $[T, d]$, where T is the number of frames to generate.

We concatenate these latent codes with per-class embeddings $e(y)$ of size 120, where y is a randomly sampled class label. This results in a sequence of T codes as $Z_F = \left[\begin{bmatrix} z_{gru}^1 \\ e(y) \end{bmatrix}, \dots, \begin{bmatrix} z_{gru}^T \\ e(y) \end{bmatrix} \right] \in \mathbb{R}^{(d+120)}$. We feed Z_F into the image generator to generate a sequence of T frames, as shown in Figure 3.

4.1.3 Image Generator

To synthesize ‘‘realistic’’ individual frames, we use the image generator of BigGAN [3], which is based on the ResNet

¹This paper uses Z_{ABC} to refer to the latent space, a vector of latent codes or a single latent code, this should be clear from the context.

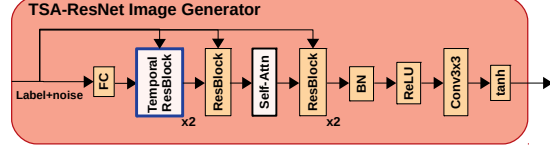
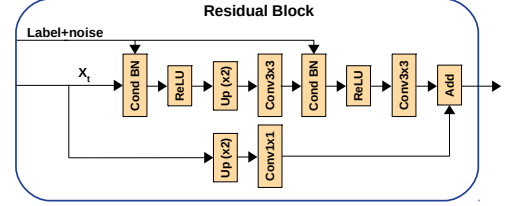
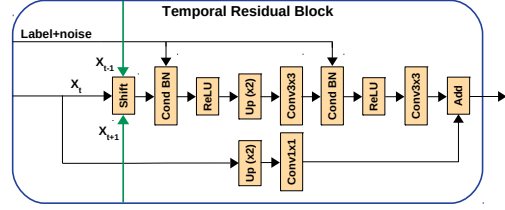


Figure 3: The Temporal Self Attention (TSA) image generator architecture. Two different variants can be applied at the first pair of residual blocks (white), these are shown in Figure 4. Note the self-attention layer (also white).



(a) Vanilla Residual Block Architecture.



(b) Temporal Residual Block Architecture.

Figure 4: Two options for the Temporal ResBlock from Figure 3. Either: (a) the vanilla or (b) the temporal variant of the residual up-sampling block can be used. The shift operation shares feature information from neighboring feature maps (X_{t-1}, X_{t+1}), shown in green. The operation $Up(x2)$ means up sampling via interpolation by a factor of 2.

architecture [27]. The image generator (G_{Image}) works independently on each frame using only 2D operations and a self-attention mechanism. In this baseline image generator, all temporal information comes from the GRU cell in the sequence generator.

Temporal Self-Attention Generator: To model temporal information already at the image generation stage, we introduce the Temporal Self-Attention (TSA) generator, which adds an implicit temporal structure to the baseline model. Compared to the vanilla version of the generator, the Temporal Self-Attention (TSA) generator not only generates based on spatial features, but also takes into account the temporal information. It does this by observing the features generated by previous and future frames with the goal of generating more temporally consistent motions. We repurpose the Temporal Shift Module (TSM), proposed by Lin *et al.* [23]. Unlike full 3D convolutions it only shares a small subset of features between neighboring frames. This allows

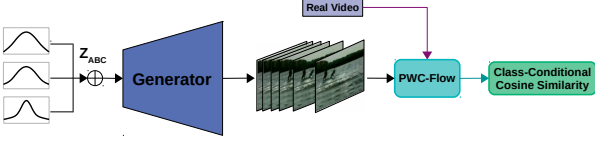


Figure 5: Illustration of the motion constraint calculation.

faster inference and uses less parameters than 3D models. In the MVT-TSA variant of the architecture, the temporal shift module can simply be added to the beginning of the residual blocks, as shown in Figure 4b. This is in contrast to the MVT variant of our architecture, which uses normal residual blocks, shown in Figure 4a. We only vary the first two residual blocks of our network, and call these *temporal residual blocks* to distinguish them from the latter two residual blocks which always use the normal variant. This is shown in Figure 4. All residual blocks use conditional batch normalization and receive as input the vector Z_F .

4.2. Discriminator

We use two independent discriminators, an image discriminator, D_{Image} , and a video discriminator named D_{Video} .

Image Discriminator D_{Image} gives a frame-wise assessment of content and structure. D_{Image} is a ResNet based architecture [27], similar to BigGAN [3], it is applied to a subset of N frames of the video. D_{Image} is doing the heavy lifting with respect to image quality. N remains a hyperparameter that allows a trade-off between memory efficiency and frame quality.

Video Discriminator D_{Video} examines the temporal consistency of videos and provides the generator with a learning signal to generate a consistent motion throughout all T frames. MVT-GAN's D_{Video} is inspired by MoCoGAN's [44] video discriminator. We chose this architecture to keep the network efficient. The factorized design allows for smaller D_{Video} networks as it can focus on the temporal aspect.

4.3. Motion Constraint

Prior work has used optical flow to generate videos by warping the images [33, 58] or using it as a prior to generate spatial features [28, 26]. We rather introduce an intra-class constraint on similarity between optical flows produced by real videos and by generated videos. An illustration is provided in Figure 5.

To estimate this constraint, first we generate synthetic samples and sample real videos from the dataset. Optical flow is calculated using the PWC Flow network [10]. Moreover, we calculate the cosine similarity between flows resulted from real videos and flows from generated videos. We only do this for pairs of real and synthetic videos with matching labels:

$$L_M = \frac{1}{C} \sum_i^B \sum_j^B \text{Sim}(f_{r_j}, f_{f_i}) = \begin{cases} \frac{f_{r_j} \cdot f_{f_i}}{\|f_{r_j}\| \cdot \|f_{f_i}\|} & \text{if } y_{f_i} = y_{r_i} \\ \emptyset & \text{otherwise} \end{cases} \quad (5)$$

where f_r and f_f stand for real and generated flows, respectively, B is batch size and C is the number of matching real and generated flow pairs. This similarity measure enforces the similarity of motion between samples from the same class. Finally, we add the constraint only to the generator loss:

$$L_{G_M} = L_G + (\alpha \cdot (1 - L_M)) \quad (6)$$

where α is a hyperparameter that controls the importance of the motion constraint L_M .

5. Symmetric Similarity Score (S3)

The past few years have seen the adoption of the Inception Score [51] (IS) as a metric to evaluate GANs. Said adoption is based on two criteria: the distribution of predicted labels $P(y | x)$ should have a low entropy and the marginal distribution $P(y) = \int_z P(y | x = G(z)) dz$ should have a high entropy. The final score is a measure of how dissimilar both distributions are from each other (Equation 7).

$$\text{IS}(G) = \exp(D_{KL}(P(y | x) | P(y))) \quad (7)$$

Although higher IS correlates with subjective quality, it is difficult to compare this score between papers. Yushchenko *et al.* [52] showed that small changes to the data pre-processing leads to a change between 7% and 17% in IS score.

There is no standardized way to compute means and standard-deviations of IS scores. This leaves it up to each research team to choose the number of samples per calculation and the number of evaluations to be performed. In addition, it is difficult to compare when different models and learning frameworks are used for label prediction.

Another point of concern is IS's vulnerability to adversarial samples [46]. Higher IS scores might lead us to believe a model is better, but this may be due to adversarial samples that fool the classifier to be confident about belonging to a particular class. Thus it obtains a higher score than it should [4].

An important criterion for GANs is whether they avoid mode collapse/memorization, which can be seen as collapse in terms of intra-class diversity. While IS does not address this [5], the Frechet Inception Distance [36] (FID) was introduced as a metric to measure both intra-class diversity and sample quality. Instead of using the distribution $P(y | x)$ of a given network, FID uses the features produced by an intermediate layer to parameterize a multi-variate normal distribution of real and fake features respec-

tively. FID rates the fake samples by calculating the distance between distributions using the Frechet distance, also known as Wasserstein-2 distance (Equation 8), the closer to each other the distributions are, the better generated samples are considered to be

$$\text{FID}(x_r, x_f) = \|\mu_r - \mu_f\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_f - 2(\Sigma_r \Sigma_f)^{\frac{1}{2}}) \quad (8)$$

where μ_r and μ_f are the mean of the fake and real feature distributions respectively, Σ_r and Σ_f are the covariances of said distributions and Tr sums all diagonal elements.

FID is able to directly compare generated samples to real samples to provide a more robust score of the synthetic data. However, it assumes the features follow a normal distribution, which is not true on real world datasets. Thus, two completely different distributions might lead to a good score while not being actually similar. FID is also vulnerable to the same pre-processing and model woes as IS, except for adversarial samples.

We introduce the Symmetric-Similarity-Score (S3), another measure of performance, which uses generalization of a classifier between real and synthetic samples to measure quality. The performance of a classifier trained on synthetic data and evaluated on real data (SeR) should increase, if synthetic samples are diverse and realistic. A classifier trained on real data being evaluated on synthetic (ReS) data should only perform well, if synthetic samples are realistic. We combine these values by computing their geometric mean after normalizing these values by the classifier performance obtained when training on real data and evaluating on real data (ReR) (Equation 9). S3 has the advantages of capturing intra-class diversity, being more robust to over-confident predictions and small changes in the model’s parameters, while still being easier to interpret than IS or FID.

$$S3 = \sqrt{\left(\frac{SeR}{ReR}\right) \cdot \left(\frac{ReS}{ReR}\right)} \quad (9)$$

6. Experiments

We tested MVT-GAN by both qualitatively and quantitatively evaluating the quality of frames, the realism of whole videos, diversity and memorization using several different datasets. FID and further qualitative samples will be found in the supplementary material.

6.1. Datasets

We use the following datasets for our experiments:

UCF-101: 13,220 videos of 101 different sports action classes, it has been adopted as benchmark for action recognition and video generation. We trained models to generate samples both at 96×96 and at 128×128 , we resize to 127×127 and 170×170 respectively and crop to its corresponding final size. We set N to 8. [30]

Weizmann: 93 videos of 9 people performing 10 different whole body actions. It was used to test viability of changes to our method. To train we cropped the videos to 96×96 . For all experiments, we randomly extract 16 subsequent frames and set N to 4. [31]

Jester: 118,562 videos and 27 different hand gesture classes. Due to the small frame size, we first re-size 96×136 before cropping to 96×96 to preserve the aspect ration. As in Weizmann, we extract 16 subsequent frames and set N to 4. [24]

6.2. Model Configurations

Three different variation of our method were tested in order to find the strongest configuration.

MVT: A model using the vanilla self-attention generator and all latent variables’ distribution were $\mathcal{N}(\mu = 0, \sigma = 1)$.

MVT-MC: Same as above, but with addition of the motion constraint to the generator with $\alpha = 1$.

MVT-VAR: A model using the vanilla generator and change the latent variables’ distribution deviations to $\sigma_A = 0.5$, $\sigma_B = 1$ and $\sigma_C = 2$.

MVT-TSA: Same as above, but we change the vanilla generator to the temporal self-attention one.

For all trained models we set d to 120 by setting d_A , d_B and d_C to 20, 20 and 80 respectively. We employ a learning rate of 5×10^{-5} for the generator and 2×10^{-4} for the discriminator. The video length T is fixed to 16. All experiments performed on Weizmann and Jester are done with a batch size of 40. We trained both MVT and MVT-TSA to generate 96×96 and 128×128 sized samples respectively. MVT was trained on a batch size of 56, while MVT-TSA was trained on a batch size of 64.

6.3. Quantitative Evaluation

A thorough evaluation of quality of samples simply by qualitative experiments is not possible due to the sheer number of samples that need to be evaluated in order to do a proper assessment about quality.

IS: We evaluate the IS as comparative benchmark on the UCF-101 dataset. The IS is calculated using the last layer of a C3D² [13] model which was pre-trained on Sports-1M [1] and fine-tuned on the UCF-101 dataset as per [37]. The model receives frames of size 128×128 , we resized when necessary. We use 10,000 samples to calculate the score. The standard deviation is calculated by repeating this procedure 10 times. Values for VGAN, MoCoGAN, Progressive VGAN, TGANv2, and DVD-GAN are shown as reported in [2], TGAN and MDP’s values are reported as they appear

²Using the code provided by github.com/pfnet-research/tgan.

Dataset	Method	Conditional	IS ($\uparrow$)
UCF-101	VGAN [6]	$\times$	8.18 ± 0.09
	TGAN [37]	$\times$	11.85 ± 0.07
	MDP [52]	$\times$	11.86 ± 0.11
	MoCoGAN [44]	$\times$	12.42 ± 0.03
	Prog. VGAN [11]	$\times$	14.56 ± 0.05
	TGANv2 [42]	$\times$	24.34 ± 0.35
	DVD-GAN* [2]	$\times$	27.38 ± 0.53
	TGAN [37]	$\checkmark$	15.83 ± 0.18
	DVD-GAN* [2]	$\checkmark$	32.97 ± 1.7
	MVT	$\checkmark$	31.91 ± 0.14
	MVT-TSA	$\checkmark$	42.79 ± 0.63
Jester	MVT-TSA	$\checkmark$	11.90 ± 0.12

Table 1: Comparison of Inception Score between models on UCF-101. (*DVD-GAN is concurrent work.)

in the original works [37, 52]. On Jester, we use a TSN [53] action recognition network pre-trained on ImageNet [9] and fine-tuned on Jester, otherwise the same procedure as for UCF-101 is used. MVT-TSA produces samples that beat the state of the art (see Table 1). Although, IS scores might suggest an overwhelming improvement over all existing methods, when qualitatively comparing samples from MVT and MVT-TSA (Figure 7) we don’t see a vast improvement as the score suggests. This could be because our samples might be exploiting C3D [13] in a way that it is over confident about its prediction, thus a higher score.

S3: To calculate S3 on Weizmann and UCF-101 we used the TSN [53] action recognition network pretrained on ImageNet [9]. Since Jester is a motion dataset we decided to use ECO [40] because it incorporates a 3D network, to improve classification. On the Weizmann dataset we compare to MoCoGAN. All experiments on a dataset were done under the same conditions. Training details of the classifier will be included in the supplemental material. From Table 2, we can see MVT-TSA produces a significant performance increase over all methods. It appears MVT-TSA, is able to increase the quality of the samples with a minimal loss of diversity. On UCF-101, Table 2 shows small discrepancies between SeR and ReS indicating a good diversity of samples, as is supported by the qualitative interpolation experiments, which do not indicate memorization (Figure 8, Figure 9). S3 scores produced by MVT-TSA show an improvement over MVT. The performance difference between methods seems more reasonable when visually comparing samples, than the one showed in Table 1. This shows S3 is more reliable than IS. To facilitate future comparisons, S3 results on a 3D ResNet-18 [29] model are provided in the supplementary material.

6.4. Qualitative Evaluation

To supplement the quantitative evaluation, we provide a qualitative evaluation focusing on the aspects that are dif-

Dataset	Method	Train on: Synth.	Real		S3
		eval. on: Real	Synth.	Real	
Weizmann	MoCoGAN	61.11	32.83	95.83	0.47
	MVT	80.31	62.53	95.83	0.73
	MVT-MC	86.11	63.84	95.83	0.77
	MVT-VAR	88.30	64.29	95.83	0.79
	MVT-TSA	88.10	71.43	95.83	0.83
UCF101	MVT	45.5	46.8	85.9	0.54
	MVT-TSA	48.55	54.91	85.9	0.60
Jester	MVT-TSA	15.85	13.86	91.4	0.16

Table 2: Evaluation S3 measure based on action classification generalization between generated and real samples.

ficult to evaluate numerically.

There is no quantitative measure of memorization in generative models, thus we check this via *intra-class interpolation*, *class interpolation* and k-NN retrieval. In *intra-class interpolation* we linearly interpolate between two different latent codes Z_{ABC} while keeping the label fixed, as shown in Figure 9. In the Figure 8, we explore *class interpolation* by linearly interpolating between label embeddings, while keeping Z_{ABC} fixed. Figures 9 and 8 show smooth transition between modes and classes. Samples from the retrieval experiment (Figure 6), show that generated samples are noticeable dissimilar to real samples, this suggests the model does not suffer from memorization.

Metrics indicate our method produces higher quality samples than previous methods. Figure 1, which compares

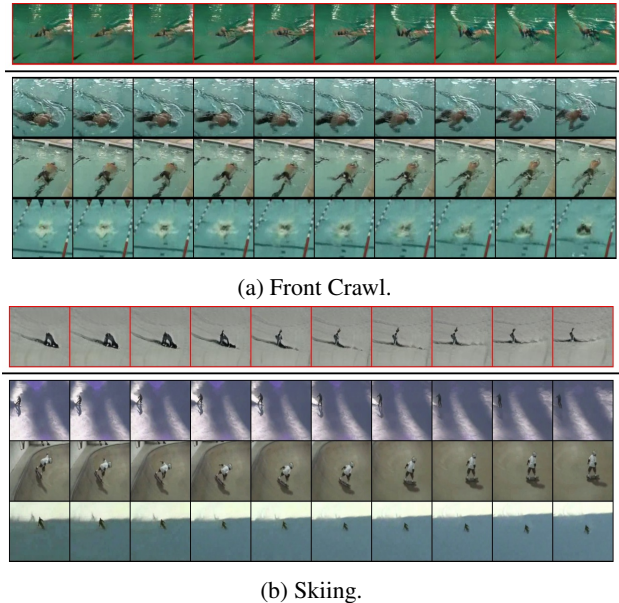


Figure 6: Examples of retrieval of top-3 nearest neighbors (black) of MVT-TSA generated samples (red). We can see that although the generated samples look similar to their respective 3-NNs, they are still quite visually distinct. This implies the model isn’t just memorizing the real data.

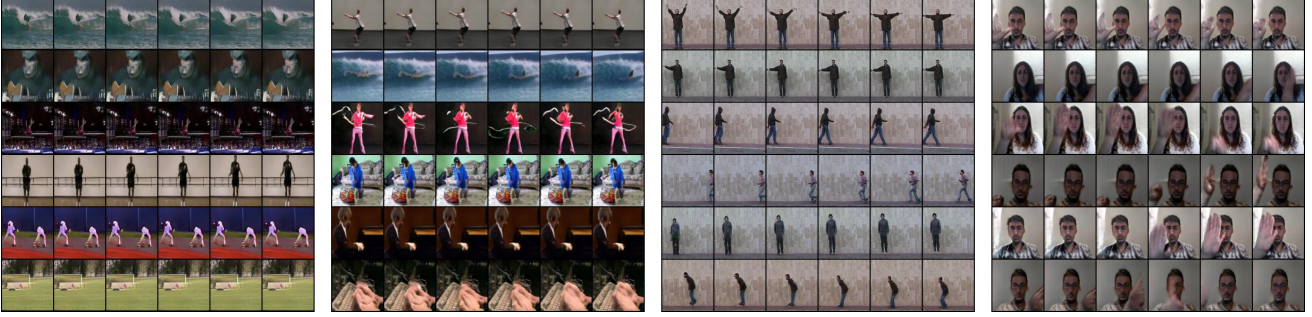


Figure 7: Generated samples (from left to right) of MVT trained on UCF-101 and MVT-TSA trained on UCF-101, Weizmann and Jester. Additional videos will be provided in the supplemental material.



(a) MVT.



(b) MVT-TSA.

Figure 8: Interpolation Performance showing smooth transitions between classes on UCF-101. Each column is a sequence. (a) The top figure is interpolating between classes writing on board (left) and pole vault (right), while the bottom one is interpolating volleyball spiking (left) and frisbee catch (right). (b) The top figure is interpolating between classes basketball (left) and frisbee catch (right), while the bottom one is interpolating between golf swing (left) and diving (right).

frames from our method against others, supports this. We also provide a supplemental video, which gives a much richer impression of the spatio-temporal quality of the generated videos than the images.

7. Conclusion

We presented a new architecture design for video generation, which achieves state-of-the-art IS performance on the UCF-101 benchmark. This architecture was obtained by



(a) MVT.



(b) MVT-TSA.

Figure 9: Example of intra-class interpolation on UCF-101. The vertical axis represents time, the horizontal axis represents different modes of the class. We sample two latent codes which are represented by the leftmost and right most samples and linearly interpolate between them to generate intermediate latent code samples.

systematically evaluating constituent components of noise generation, sequence generation, and frame generation. It allows training on the challenging Jester dataset, not previously used in video generation. As a supplement to the well established IS score, we proposed the generalization based S3 score, which is intended to be sensitive to intra-class variation. On this metric, too, our method achieves the best results. These quantitative results correspond to our qualitative evaluation where we do not see any evidence of memorization.

References

- [1] G. Toderici R. Sukthankar T. Le-ung A. Karpathy, S. Shetty and Li Fei-Fei. Large-scale video classification with convolutional neural networks. *CVPR*, 2014.
- [2] Jeff Donahue Aidan Clark and Karen Simonyan. Adversarial video generation on complex datasets. *arXiv:1907.06571*, 2019.
- [3] Jeff Donahue Andrew Brock and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. *ICLR*, 2019.
- [4] Shane Barratt and Rishi Sharma. A note on the inception score. *arXiv:1801.01973*, 2018.
- [5] Ali Borji. Pros and cons of GAN evaluation measures. *CoRR*, abs/1802.03446, 2018.
- [6] Hamed Pirsiavash Carl Vondrick and Antonio Torralba. Generating videos with scene dynamics. *NeurIPS*, 2016.
- [7] João Carreira and Andrew Zisserman. Quo vadis, action recognition? A new model and the kinetics dataset. *CoRR*, abs/1705.07750, 2017.
- [8] Ian Goodfellow Chelsea Finn and Sergey Levine. Unsupervised learning for physical interaction through video prediction. *NeurIPS*, 2016.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [10] Ming-Yu Liu Deqing Sun, Xiaodong Yang and Jan Kautz. Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume. *CVPR*, 2018.
- [11] Danda Pani Paudel Dinesh Acharya, Zhiwu Huang and Luc Van Gool. Towards high resolution video generation with progressive growing of sliced wasserstein gans. *arXiv:1810.02419*, 2018.
- [12] Oscar Tckstrm Dirk Weissenborn and Jakob Uszkoreit. Scaling autoregressive video models. *arXiv:1906.02634*, 2019.
- [13] Rob Fergus Lorenzo Torresani Du Tran, Lubomir Bourdev and Manohar Paluri. Learning spatiotemporal features with 3d convolutional networks. *ICCV*, 2015.
- [14] Rajesh Ranganath Dustin Tran and David M. Blei. Hierarchical implicit models and likelihood-free variational inference. *NeurIPS*, 2017.
- [15] C. Han G. Kwon and D. Kim. Generation of 3d brain mri using auto-encoding generative adversarial networks. 2019.
- [16] Y.-W. Tai H. Cai, C. Bai and C.-K. Tang. Deep video generation, prediction and completion of human action sequences. *ECCV*, 2018.
- [17] Dimitris Metaxas Han Zhang, Ian Goodfellow and Augustus Odena. Self-attention generative adversarial networks. *PMLR*, 2019.
- [18] Kensho Hara, Hirokatsu Kataoka, and Yutaka Satoh. Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet? In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6546–6555, 2018.
- [19] DM. Hawkins. The problem of overfitting.
- [20] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, Günter Klambauer, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a nash equilibrium. *CoRR*, abs/1706.08500, 2017.
- [21] Mehdi Mirza Bing Xu-David Warde-Farley Sherjil Ozair Aaron Courville Ian J. Goodfellow, Jean Pouget-Abadie and Yoshua Bengio. Generative adversarial nets. *NeurIPS*, 2014.
- [22] J. Marino G. Mori J. He, A. Lehrmann and L. Sigal. Probabilistic video generation using holistic attribute control. *ECCV*, 2018.
- [23] Chuang Gan Ji Lin and Song Han. Tsm: Temporal shift module for efficient video understanding. *ICCV*, 2019.
- [24] Ingo Bax Joanna Materzynska, Guillaume Berger and Roland Memisevic. The jester dataset: A large-scale video dataset of human gestures. *ICCV*, 2019.
- [25] Honglak Lee Richard Lewis Junhyuk Oh, Xiaoxiao Guo and Satinder Singh. Action-conditional video prediction using deep networks in atari games. *NeurIPS*, 2015.
- [26] Xu Jia Jing Shao Lu Sheng-Junjie Yan-Xiaogang Wang. Junting Pan, Chengyu Wang. Video generation from single semantic label map. *CVPR*, 2019.
- [27] Shaoqing Ren Jian Sun Kaiming He, Xiangyu Zhang. Deep residual learning for image recognition. *CVPR*, 2016.
- [28] Yoshitaka Ushiku Katsunori Ohnishi, Shohei Yamamoto and Tatsuya Harada. Hierarchical video generation from orthogonal information: Optical flow and texture. *AAAI*, 2017.
- [29] Hirokatsu Kataoka Kensho Hara and Yutaka Satoh. Learning spatio-temporal features with 3d residual networks for action recognition. *ICCV*, 2017.
- [30] Amir Roshan Zamir Khuram Soomro and Mubarak Shah. A dataset of 101 human actions classes from videos in the wild. *arXiv:1212.0402*, 2012.
- [31] E. Shechtman M. Irani L. Gorelick, M. Blank and R. Basri. Actions as space-time shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2007.
- [32] J. H. Lim and J. C. Ye. Geometric gan. *arXiv:1705.02894*, 2017.
- [33] Jiaming Guo Jing Shao Xiaogang Wang-Chen Change Loy. Lu Sheng, Junting Pan. Unsupervised bi-directional flow-based video generation from one snapshot. *arXiv:1903.00913*, 2019.
- [34] Mario Lucic, Karol Kurach, Marcin Michalski, Sylvain Gelly, and Olivier Bousquet. Are gans created equal? a large-scale study. 11 2017.
- [35] Lon Bottou. Martin Arjovsky, Soumith Chintala. Wasserstein generative adversarial networks. *PMLR*, 2017.
- [36] Thomas Unterthiner Bernhard Nessler Martin Heusel, Hubert Ramsauer and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *NIPS*, 2017.
- [37] andShunta Saito. Masaki Saito, Eiichi Matsumoto. Temporal generative adversarial nets with singular value clipping. *ICCV*, 2017.
- [38] Camille Couprie Michael Mathieu and Yann LeCun. Deep multi-scale video prediction beyond mean square error. *ICLR*, 2016.
- [39] Takeru Miyato and Masanori Koyama. cgans with projection discriminator. *ICLR*, 2018.

- [40] Kamaljeet Singh Mohammadreza Zolfaghari and Thomas Brox. Eco: Efficient convolutional network for online video understanding. *ECCV*, 2018.
- [41] Karen Simonyan Oriol Vinyal Alex Gravesa Nal Kalchbrenner, Aaaron van den Oord and Koray Kavukcuoglu. Video pixel networks. *ICML*, 2017.
- [42] Masaki Saito and Shunta Saito. Tganv2: Efficient training of large models for video generation with multiple subsampling layers. *arXiv:1811.09245*, 2018.
- [43] Tim Salimans, Ian J. Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. *CoRR*, abs/1606.03498, 2016.
- [44] Xiaodong Yang Sergey Tulyakov, Ming-Yu Liu and Jan Kautz. Mocogan: Decomposing motion and content for video generation. *CVPR*, 2018.
- [45] Ravi Kiran Sarvadevabhatla Swaminathan Gurumurthy and R. Venkatesh Babu. Deligan: Generative adversarial networks for diverse and limited data. *CVPR*, 2017.
- [46] Zaremba W. Sutskever I. Bruna J. Erhan D. Goodfellow I. J. Szegedy, C. and R. Fergus. Intriguing properties of neural networks. *ICLR*, 2014.
- [47] Masanori Koyama Takeru Miyato, Toshiki Kataoka and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *ICLR*, 2018.
- [48] Samuli Laine Tero Karras and Timo Aila. A style-based generator architecture for generative adversarial networks. *CVPR*, 2019.
- [49] Timo Aila Samuli Laine Tero Karras, Tero Karras and Jaakko Lehtinen. Progressive growing of gans for improved quality stability and variation. *ICLR*, 2018.
- [50] Katherine L. Bouman Tianfan Xue, Jiajun Wu and William T. Freeman. Visual dynamics: Probabilistic future frame synthesis via cross convolutional networks. *NeurIPS*, 2016.
- [51] Wojciech Zaremba Vicki Cheung Alec Radford Tim Salimans, Ian Goodfellow and Xi Chen. Improved techniques for training gans. *NIPS*, 2016.
- [52] Nikita Araslanov Vladyslav Yushchenko and Stefan Roth. Markov decision process for video generation. *ICCV*, 2019.
- [53] Xiong Y. Wang Z. Qiao Y. Lin D. Tang X. Gool L.V. Wang, L. Temporal segment networks for action recognition in video. *TPAMI*, 41(11):2740 – 2755, 2019.
- [54] Dongze Lian Wen Liu, Weixin Luo and Shenghua Gao. Future frame prediction for anomaly detection a new baseline. *CVPR*, 2018.
- [55] Rein Houthoofd John Schulman Ilya Sutskever Xi Chen, Yan Duan and Pieter Abbeel. Infogan: Interpretable representation learning by information maximizing generative adversarial nets. *NeurIPS*, 2016.
- [56] Dinghan Shen David Carlson Lawrence Carin. Yitong Li, Martin Renqiang Min. Video generation from text. *AAAI*, 2018.
- [57] Seyed shahabeddin Nabavi Yiwei Lu, Mahesh Kumar Krishna Reddy and Yang Wang. Future frame prediction using convolutional vrnn for anomaly detection. *AVSS*, 2019.
- [58] X Huang Z Hao and S Belongie. Controllable video generation with sparse trajectories. *CVPR*, 2018.

A. Architectural Details

We adopted most of BigGAN’s [3] architectural choices in G_{Image} , with the exception that we moved the self-attention module down one level of abstraction to save video memory. D_{Image} follows exactly the discriminator guidelines set in BigGAN, while D_{Video} adopted the exact architecture used in MoCoGAN [44], but extended for class conditional hinge loss per [39]. To describe the width of all networks we use the product of a layer-wise constant c and a per-layer constant a . In all experiments a was set to 96. We chose c to be [16, 8, 4, 2, 1] for G_{Image} , [1, 2, 4, 8, 16, 16] for D_{Image} and [1, 2, 4, 8] on D_{Video} .

At the input of G_{Image} we have a fully connected layer which applies an affine transformation to Z_F to transform it from $[T, d + 120]$ to $[T, w \cdot h \cdot 16 \cdot a]$. When generating 96×96 sized samples we set w and h to 3 and when we generated samples of size 128×128 they were both set to 4.

The sequence generator is composed of a fully connected layer FC and a GRU cell. FC has a size of d and the GRU cell has a size of 2048

B. Additional Experiments

B.1. S3

To calculate S3 we train a classifier on real samples and on fake samples as explained on Section 5. We calculated the S3 metric, for UCF-101 [30], based on two different classifiers, a TSN [53] action recognition network and a 3D ResNet-18 [29]. The TSN was trained on a batch size of 14 and an initial learning rate of 0.01. We trained the 3D ResNet-18 using a batch size of 32, an initial learning rate of 0.001 and a dropout probability of 0.6. To get the S3 score for Jester [24] we decided to use the ECO [40] action recognition network. We trained it using a batch size of 14, an initial learning rate of 0.001 and a dropout probability of 0.6. All networks were trained on 16 frames and their respective learning rates were scheduled to drop by an order of magnitude after failing to beat the best recorded test accuracy for 4 straight epochs.

We calculated the S3 with two different architectures for the UCF-101 dataset to provide a reference for the comparisons in the future works. Table 3 shows that the change of architecture does alter the relative performances of ReS and SeR to ReR significantly enough to produce important changes in the score. Therefore, S3 scores obtained from different classification architectures does not provide a fair comparison.

B.2. FID

FID [20] calculations were done using the features from the second-to-last layer of a TSN pretrained on Imagenet [9] and finetuned on the respective dataset it is going to be tested on. The network was trained as explained above. We

Classifier Architecture	Method	Train on: Synth.	Real		S3
		eval. on: Real	Synth.	Real	
TSN	MVT	45.5	46.8	85.9	0.54
	MVT-TSA	48.55	54.91	85.9	0.60
3D ResNet18	MVT	36.63	28.83	76.82	0.42
	MVT-TSA	44.36	29.61	76.82	0.47

Table 3: UCF-101 results of S3 on two different architectures.



Figure 10: Example of class interpolation showing smooth transitions between classes on UCF-101. Each column is a sequence.



Figure 11: Example of intra-class interpolation on UCF-101. The vertical axis represents time, the horizontal axis represents different modes of the class.

Method	Dataset	FID
MVT	UCF-101	3108.77 $\pm$ 0.04
MVT-TSA	UCF-101	3110.29 $\pm$ 0.10
	Jester	841.08 $\pm$ 0.005

Table 4: FID scores on Jester and UCF-101.

calculated FID using 4000 samples, we repeated the process 5 times to get the standard deviation. Table 4 seems to suggest MVT is better than MVT-TSA, but this could be due to the fact that FID cannot separate image quality from diversity. If we take into account IS and S3 we can deduce that although FID points to MVT being better than MVT-TSA this is most likely due to better sample diversity, not sample quality.

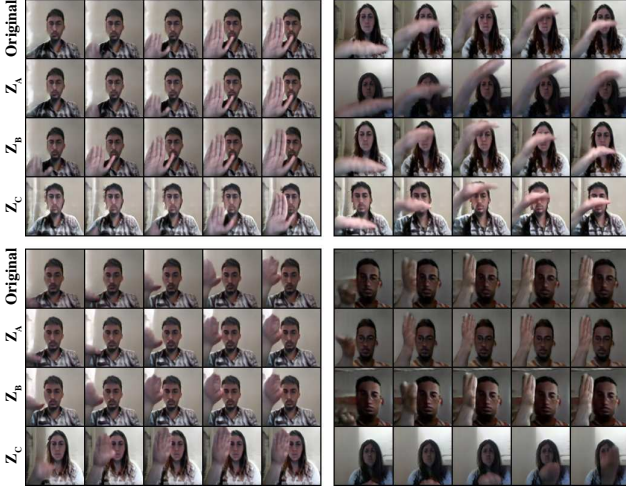


Figure 12: Latent variable experiment. We freeze two out of the three subspaces and re-sample the remaining one to produce a new sample. We compare each sample to the original to see what meaning is the network assigning to that specific subspace.

B.3. Qualitative Samples

Figure 13 shows the improvement between models in different classes of the Weizmann [31] dataset. We can clearly see an increase of generation quality, specially in fine structures, of MVT-TSA over all other methods.

Our MVT-TSA trained on Jester did not record a good performance on the S3 measure, hence we need a qualitative evaluation to look for a possible reason why this was the case.

In order to further show the models lack memorization we present more *inter-class interpolation* and *class interpolation* samples (Figure 11 & 10). Figure 11 shows a smooth transition between modes while, Figure 10 shows effortless change in class semantics.

We wanted to know if in fact using a multi-variate model for the latent codes had any effect on what the network learned. Specifically we wanted to see if assigning a different variance to each subspace had any effect on the features the network learned to map to each one of the subspaces. To test this we froze two out of the three subspaces and re-sampled the remaining one to produce a new sample. Every subspace will get a turn at being re-sampled. Figure 12 shows some examples of this experiment compared to a sample produced by the originally sampled latent vector. The samples show that the network learns to assign Z_C features that result in bigger changes in the overall visual features, like gender. We can observe as well that Z_B appears to be in charge more of motion features, without affecting features such as location or person identity as much. It appears that Z_A is in charge of more infrequent features like location

or small changes in appearance. This experiment points towards the variance assigned to a subspace being directly related the types of features it represents.



Figure 13: Generated samples (from top to bottom) of MVT , MVT-MC, MVT-VAR and MVT-TSA trained on Weizmann.